Performance & Security Report

Zamzam Fashion — .NET 10 API · v2 after hardening and a performance pass

Prepared 8 November 2026 · branch `feature/phase-a-hardening` (uncommitted) on top of `main` a070cad · compare with v1 of 7 November

> **Scope and method.** Re-review of the backend after Phase A (photo memory, caching, compression, pool cap, Dependabot, nightly backups) and a measurement-driven performance pass. Evidence: the full unit suite, CI's integration tests on real PostgreSQL, the API running locally against the Supabase dev database (headers, limits, query counts per endpoint from EF's SQL log, 25-request latency samples), an A/B run that switched builds and proved which one ran by its query count, and dependency scans. Passive checks only. **Not measurable here:** the production host (deploy is planned last), Core Web Vitals (frontend still on mock data). Latency is from a laptop in Egypt to Frankfurt and the network was ~2.4× slower than on 7 November, so compare ratios, not v1's absolute times. CHECK items list their confirming tool in Section 4; Section 6 is a **modelled calculation**.

Status key: **OK** observed and sound · **CHECK** needs access or a step not available yet · **FIX** still open.

Summary

Every backend gap from v1 that could be fixed in code is fixed and verified: the API now refuses to start on a server with a missing setting, logs quietly, caches and compresses public reads (−70–76% bytes), makes fewer database round trips on the busiest pages (−17% to −31% p50, measured), decodes phone photos at the size it needs (~10 MB instead of ~160 MB), caps its connection pool, and has a nightly encrypted backup ready. **Two items still block a real launch**: emails are log-only (customers can't approve quotes) and the frontend's proxy secret. Everything else open is a deploy-day check.

Area	Finding	Status
Security headers	nosniff, DENY, no-referrer, CSP `default-src 'none'`, CORP, **X-Robots-Tag: noindex** (new), HSTS outside dev	**OK**
Auth, CSRF, admin lock	HttpOnly+Secure+SameSite=Strict cookie; CSRF header live 400; anonymous admin live 401; lockout 5/15 min	**OK**
Rate limits	Live: 31st tracking call → 429 with Retry-After; X-Forwarded-For only trusted with the proxy secret	**OK**
Server settings	API refuses to start in Production without proxy secret, real AllowedHosts, https Site URL, storage keys (tested)	**OK**
Logging volume	Serilog overrides: 45 log lines for ~130 requests (was 4,246)	**OK**
Caching + compression	Products cached 60 s, other public reads 300 s; brotli on public reads only: −73 to −75% bytes	**OK**
Database round trips	Product list 4 → 3 queries, category page 5 → 3 (A/B measured −17% / −31% p50)	**OK**
Photo processing memory	JPEG/WebP decoded scaled (~10 MB for a 40 MP photo, was ~160 MB); 16 MP in-memory cap; one buffer	**OK**
Connection pool	Capped at 10 (Database:MaxPoolSize) unless the connection string sets its own	**OK**
Backups	Nightly encrypted `pg_dump` workflow + restore guide written; needs `BACKUP_PASSPHRASE` and a first run	**CHECK**
Emails	Still log-only (parked): customers get no confirmation and can't approve a tailoring quote	**FIX**

Area	Finding	Status
Real client IP behind Vercel	API side done; frontend `src/proxy.ts` still to add at deploy	**FIX**
Production host	.NET 10 on MonsterASP, HTTPS, IIS headers: verified only at the final deploy	**CHECK**

Test results

Test	Result	Status
Unit tests (`dotnet test` , 155 incl. 4 new)	155 passed, 0 failed (domain, application, images, catalog)	**OK**
Integration tests on real PostgreSQL (GitHub CI)	4 passed on `main` : last-piece race, 10 buyers / 3 pieces never oversell, order sequence, schema	**OK**
Smoke test against Supabase (owner run, 7 Nov)	44 / 44 checks: login, catalog CRUD, real photo upload, order flow, tailoring, quotes, settings	**OK**
Production-mode start (7 Nov)	Nothing set → refuses with 4 named problems; all set → starts, http→https 307, wrong Host → 400	**OK**
New tests for this pass	Huge JPEG scaled to a full 1600 px file; huge PNG → 413; caller buffer read in place; card shows main photo / none	**OK**
Live checks today (dev API on Supabase)	health Healthy/Healthy/Healthy; headers; br encoding; cache; 429; CSRF 400; admin 401	**OK**
A/B on the product list (same network, alternating builds)	Query count proved which build ran: 4/5 → 3/3; p50 770 → 643 ms, 935 → 640 ms	**OK**
Workflow files	YAML of backup, Dependabot and keep-alive parsed without errors; keep-alive ran green on GitHub	**OK**
Dependency scans	NuGet vulnerable incl. transitive: 0. npm (frontend): 0 critical, 0 high, 8 moderate (minimatch, dev tooling)	**OK**
Backup workflow end to end	Can only run on GitHub after merge + `BACKUP_PASSPHRASE` secret	**CHECK**

1. Performance

Before → after (this pass)

Metric	Before	After
Product list: queries per request	4	**3**
Category page: queries per request	5	**3**
Product list p50 (A/B, PowerShell client)	770 ms	**643 ms** (−17%)
Category page p50 (A/B, PowerShell client)	935 ms	**640 ms** (−31%)
Tailoring catalog transfer	17,741 B	**4,845 B** brotli (−73%)
Product list (20) transfer	3,792 B	**912 B** (−76%)
Product page transfer	1,720 B	**513 B** (−70%)
Delivery zones transfer	3,085 B	**882 B** (−71%)
Product list / page browser + CDN cache	none	**60 s**
Log lines for a short test run	4,246	**45**
Pixels held for a 40 MP JPEG (calculation)	~160 MB	**~10 MB**

A/B: the old and new query code were built and run alternately on the same network; the logged query count per request proved which build served each sample. Absolute times include ~150 ms of PowerShell overhead per request.

Latency today (curl, 25 requests each, laptop in Egypt → Supabase Frankfurt)

Endpoint	p50	p95	DB queries
`/api/categories`	157 ms	165 ms	1
`/api/products?pageSize=12`	468 ms	536 ms	3
`/api/products?…&category=dresses`	470 ms	670 ms	3
`/api/products/{slug}`	477 ms	649 ms	3
`/api/tailoring/catalog`	323 ms	507 ms	2

Raw TCP connect to the Supabase pooler measured **137–439 ms** today (about 60 ms on 7 November): the network, not the code, dominates. On a European server next to Frankfurt each round trip drops to a few ms, so the same endpoints should land far under 150 ms. That is why the pass cut **round trips** rather than SQL time.

What changed

- **Fewer round trips** (`PublicCatalog.cs`): the category filter is now a subquery, and each card's main photo joins into the page query. Variants stay a separate batch: merging them would need EF's relational package in the Application layer.
- **Cache + compression** (`CatalogController.cs`, `Program.cs`): 60 s on products (checkout re-checks stock and price), brotli/gzip only on public catalog reads, so no response carrying a tracking token or owner data is compressed.
- **Photo memory** (`SkiaImageProcessor.cs`): scaled decoding for JPEG/WebP, 16 MP in-memory cap (PNG can't scale), the upload buffer read in place.
- **Pool cap** (`DependencyInjection.cs`): Npgsql's default of 100 lowered to 10 unless set explicitly.

Targets

Metric	Good	Measured now	Where
API p95, product list (from the server's region)	≤ 300 ms	536 ms from Egypt (network-bound)	curl ×30 on the server
LCP / INP / CLS	≤ 2.5 s / 200 ms / 0.1	—	PageSpeed Insights after integration
First-load API JSON	small	3.3 KB (was 9.5 KB)	DevTools Network

2. Security

Observation	Risk	Status
All responses: `nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `CSP default-src 'none'`, `CORP same-site`, `X-Robots-Tag: noindex, nofollow`; HSTS outside dev	JSON-only API, never indexed	**OK**
Startup checks: proxy secret, AllowedHosts, https Site URL, storage keys, data-protection certificate (no DPAPI fallback)	A forgotten setting fails the deploy, not the shop	**OK**
HTTPS redirection outside dev (port 443); IIS `web.config` hides `Server` / `X-Powered-By`	Verified on the real host at deploy	**CHECK**
Compression limited to public catalog GETs	No BREACH-style size leaks on token or admin responses	**OK**
Cookie, CSRF, lockout, per-IP limits, upload checks, private photo bucket with signed links	Unchanged from v1, re-verified live	**OK**
Photo bombs: 40 MP header limit + 16 MP decode limit + 5 MB body limit	One request can no longer exhaust the host's memory	**OK**
Backups encrypted with AES-256 before leaving the runner; 7-day retention	A leaked artifact is unreadable without the passphrase	**CHECK**
Dependabot weekly (NuGet, npm, Actions), grouped minor/patch	New CVEs arrive as PRs	**OK**
Owner still on the seed password (`mustChangePassword`)	Change before launch	**FIX**
Frontend: no security headers, no `proxy.ts` yet	Teammate, at deploy (code in v1 report)	**FIX**

Still to paste at deploy (frontend, teammate)

```
// frontend/src/proxy.ts — forwards /api with the shared secret and the real client IP
import { NextResponse, type NextRequest } from "next/server";
export function proxy(request: NextRequest) {
  const url = new URL(request.nextUrl.pathname + request.nextUrl.search, process.env.API_URL);
  const headers = new Headers(request.headers);
  headers.set("X-Proxy-Secret", process.env.PROXY_SHARED_SECRET!);
  headers.set("X-Forwarded-For", request.headers.get("x-forwarded-for") ?? "");
  return NextResponse.rewrite(url, { request: { headers } });
}
export const config = { matcher: "/api/:path*" };
```

3. SEO and sharing (quick check)

Item	Note	Status
API never indexed (`X-Robots-Tag`)	Done this pass	**OK**
Storefront title, description, Open Graph, canonical, sitemap, alt text	Frontend on mock data — check after integration	**CHECK**

4. How to verify the CHECK items

Goal	Tool / command
Backup works	GitHub → Actions → Database backup → Run workflow → green + `zamzam-YYYY-MM-DD` artifact; decrypt once with `gpg --decrypt`
Production host	`curl -sI https://zamzam-fashion.runasp.net/health` → 200, no `Server` / `X-Powered-By`; `curl -sI http://…` → 307/308 to https
OpenAPI hidden in production	`curl -s -o NUL -w "%{http_code}" https://zamzam-fashion.runasp.net/openapi/v1.json` → 404
Server / DB proximity	From the server: `/api/products?pageSize=12` ×30 → expect well under 150 ms p50 when the server is in Europe
Real client IP	After `proxy.ts`: 31 tracking calls from one network → 429 only for that network
Core Web Vitals	pagespeed.web.dev on the Vercel URL (mobile + desktop) after integration
Headers / TLS	securityheaders.com, developer.mozilla.org/observatory, SSL Labs on both hosts
Capacity	`k6 run -e BASE_URL=https://… docs/reports/zamzam-api-loadtest.js` on staging, off-peak

5. Priority action list

1. **Merge Phase A** (this branch) and add the `BACKUP_PASSPHRASE` secret; run the backup workflow once. Confirm: green run + artifact.
2. **Decide the email provider** (domain → Resend, none → Brevo) before launch: quote approval depends on the emailed link. Confirm: a test order emails the tracking link.
3. **Change the owner password** (still the seed password). Confirm: `/api/auth/me` → `mustChangePassword: false`.
4. **Frontend: compress the 656 KB navbar logo** to WebP/SVG and use `srcSet` + lazy loading for product photos (teammate). Confirm: PageSpeed LCP ≤ 2.5 s mobile.
5. **Frontend: add `src/proxy.ts`** and the security headers block at deploy. Confirm: securityheaders.com A, per-network 429.
6. **At deploy**: separate production Supabase project, pick an EU server close to Frankfurt, run the k6 script. Confirm: §4 rows.

6. Capacity estimate (modelled)

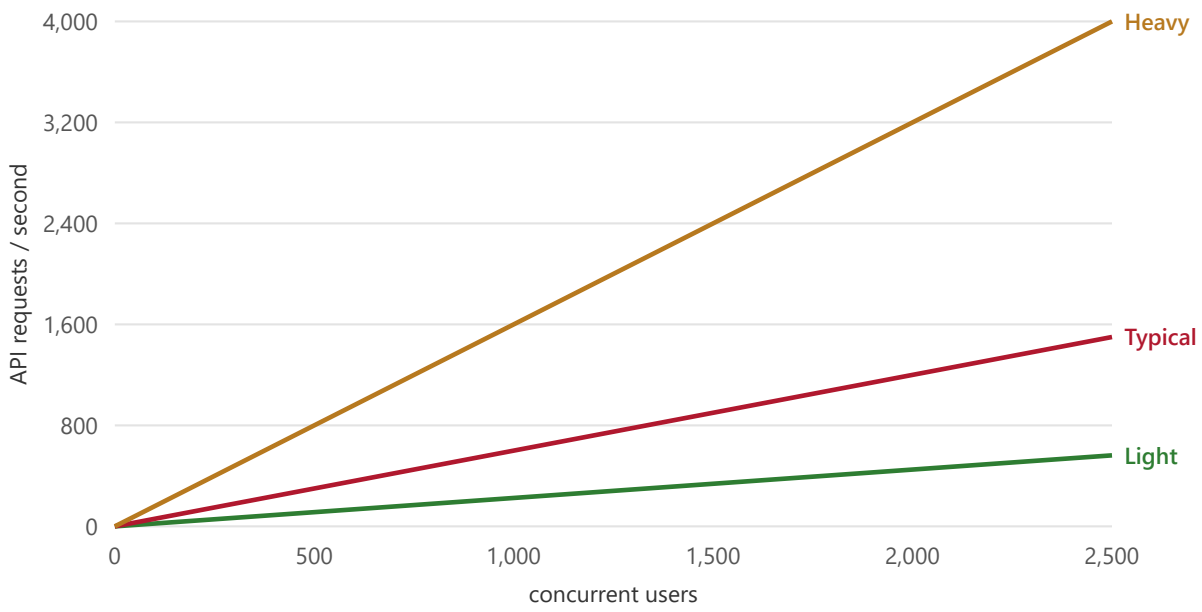
This section is a calculation, not a measurement.

No load was generated. API requests only: photos come from Supabase's CDN and pages from Vercel.

```
requests/sec = (concurrent users / read time in seconds) x API requests per page view
visits/sec   = concurrent users / read time (typical, 5 s)
bandwidth    = visits/sec x first-visit MB x 8   (Mbps)
transfer/hr  = visits/sec x 3600 x first-visit MB / 1024   (GB)
uncached list capacity = pool size / seconds per list = 10 / 0.468
```

Scenario	API requests / view	Read time	Reasoning
Light	1.8	8 s	Cached settings/zones, visitor skims a few pages
Typical	3	5 s	Product list or detail + settings + one cached lookup per page
Heavy	4.8	3 s	Cold cache, fast filtering and paging, estimate calls

API requests per second against concurrent users (calculation, not measured)



Concurrent users	Light (req/s)	Typical (req/s)	Heavy (req/s)
25	5.6	15.0	40.0
100	22.5	60.0	160.0
250	56.3	150.0	400.0
500	112.5	300.0	800.0
1,000	225.0	600.0	1,600.0
2,500	562.5	1,500.0	4,000.0

Bandwidth check (calculation)

Users	Visits / s	API Mbps (3.3 KB, measured, compressed)	Full page Mbps (1.5 MB assumed)	Full page GB / hour
100	20.0	0.52	240.0	105.5

Users	Visits / s	API Mbps (3.3 KB, measured, compressed)	Full page Mbps (1.5 MB assumed)	Full page GB / hour
500	100.0	2.61	1,200.0	527.3
1,000	200.0	5.23	2,400.0	1,054.7

How to read this

With a pool of 10 connections and 468 ms per uncached list from Egypt, one instance serves about **21 uncached product lists per second** (calculation). From a European server the time per list falls several times, and with the 60 s cache most list requests never reach the database at all. API bandwidth is negligible after compression (5.23 Mbps at 1,000 users); full-page traffic lands on Vercel and Supabase's CDN, so check their plans' transfer quotas before a campaign. Checkout and tailoring requests are the expensive calls and are rate-limited. The k6 script from v1 (`docs/reports/zamzam-api-loadtest.js`) still applies: run it on staging, off-peak, after `proxy.ts`.

Assumptions: 3 API calls per page view (Typical), 0.6× / 1.6× for Light / Heavy; read times 8 / 5 / 3 s; API first visit 3,426 B measured with brotli (categories and settings counted uncompressed); full page 1.5 MB assumed (storefront not connected yet); list time = today's measured p50.
Limitations: findings come from the source code and the API running locally at the time of review; no intrusive or active security testing was performed, and nothing was measured on the production host. Update with the PageSpeed run, `curl -sI` from MonsterASP and the k6 summary.

Appendix — performance-peak audit (31 techniques)

✅ applied · ⚠️ needs improvement · ❌ not applied · ➖ not applicable · CHECK needs the deployed or connected app. Frontend rows are audit-only (teammate's code).

Technique	Status	Evidence
1 Skeleton loading	➖ / CHECK	Frontend on mock data; prompt requires loading states per screen
2 WebP / AVIF	✅	API makes WebP 400/800/1600 + srcSet; frontend still plain `<img>` + JPG/PNG assets
3 Duplicate API calls	➖	Frontend not connected yet
4 Debounce	⚠️	Prompt asks for 400 ms debounce on the tailoring estimate
5 Pagination	✅	`Paging.Normalize`, max 100, on every list
6 CDN	✅ / CHECK	Photos on Supabase CDN; pages on Vercel at deploy
7 Background jobs	✅	Emails via queue + hosted service; photo work must finish in the request (its result is the response)
8 N+1 queries	✅	Batched/split queries; counted per endpoint (1–3)
9 Brotli / Gzip	✅ *(applied)*	Public catalog reads only (BREACH-safe), −70–76%
10 Redis	➖	Small catalog, 60–300 s HTTP cache is enough; would add infra for no gain
11 Unused dependencies	✅	Backend minimal; frontend: next, react, zustand only
12 API timeouts	✅	Storage HttpClient 60 s; Npgsql connect/command defaults; CancellationToken everywhere
13 AI streaming	➖	No AI features in phase 1
14 Optimistic UI	➖	Frontend; orders must stay server-confirmed
15 Lazy loading	⚠️	Frontend images not lazy (teammate)
16 Parallel requests	⚠️	One DbContext can't run queries in parallel; fewer round trips chosen instead
17 DB indexes	✅	Products (category, visible, deleted), images (product, sort), variants unique, orders (status, date), number, phone
18 OpenTelemetry	❌	Not needed at this size; Sentry at launch covers errors
19 Server / DB proximity	CHECK	Network to Frankfurt 137–439 ms today from Egypt; MonsterASP site is EU: confirm country at deploy
20 SSR / static generation	➖ / CHECK	Frontend decision (ISR for shop pages recommended)
21 Third-party scripts	✅	None
22 Code splitting	CHECK	Single client page today; routes added by the integration
23 Virtualization	➖	Lists are paged
24 Required columns only	✅	All reads project to DTOs; no full-entity loads on public lists

Technique	Status	Evidence
25 Connection pooling	✅ *(applied)*	Supabase pooler + Npgsql pool capped at 10
26 Re-renders	➖	Frontend, after integration
27 Browser caching	✅ *(applied)*	Cache-Control on every public read; no-store on health
28 Load balancing	➖	One small instance is enough (see §6)
29 Font optimization	⚠️	`next/font` with swap ✓, but 3 families / 13 weights (teammate)
30 Real user monitoring	❌	Add with Vercel Analytics at launch
31 Performance testing	⚠️	k6 script ready (`docs/reports`); run on staging/production before launch

Not applied, and why

- **Redis / load balancer / OpenTelemetry:** new infrastructure for problems this shop doesn't have; HTTP caching and Sentry cover it.
- **Single-query product page (3 → 1):** needs the relational EF package in the Application layer; revisit only if production latency demands it.
- **Compression on every response:** skipped on purpose for token and admin responses (BREACH).